

GHA on-boarding user guide

This document illustrates step by step process to on-board any repo/micro-service to GHA CD.

<Example snippets correspond to **cvs-health-source-code** repos, but its the same for **cvs-health-pcw-source-code** repos too, its just that the org name has to be changed.>

⚠ Please review all the steps and then start with the implementation

⚠ For teams migrating from Harness, do not use the existing manifest repo.

The following are the steps to be completed in order to on-board any repo/micro-service to GHA CD

1. Creating **.github/workflows/cd.yaml** file at the root.
2. Creating **deploy-configs** folder to host **env./namespace** specific helm **values/configs** for the deployment.
3. Creating manifest repo if does not exists along with cutting out a branch on the namespace name.
4. Creating **namespace-configs/<namespace>.yaml** at the respective branch in the manifest repo.
5. Setting up Argo credentials as Repository secrets and Variables.
6. Setting up environments with cluster name, type and manifest repo details.
7. Trigger the deployment.

REVIEW THE FOLLOWING STRUCTURES:

YOUR APPLICATION REPOSITORY:

```
1 | your-application-repo/
2 |   ├── .github/
3 |   |   └── workflows/
4 |   |       └── cd.yaml                # CD workflow (copy from
                                         # Your application source code
5 |   |
6 |   └── deploy-configs/
7 |       ├── [common]/                # e.g., dev, staging, qa
8 |       |   └── common.yaml
9 |       |
10 |      ├── [namespace1]/             # e.g., dev, staging, qa
11 |      |   └── values/
12 |      |       ├── v1.yaml           # Helm values version 1
13 |      |       └── v2.yaml           # Helm values version 2
14 |      (optional)
15 |      ├── config/
16 |      |   ├── config1.yaml          # configs to be mounted
17 |      |   └── config2.json
18 |      └── [namespace2]/             # e.g., prod
19 |          ├── values/
20 |          |   └── v1.yaml            # Helm values version 1
21 |          └── config/
22 |              ├── config1.yaml      # configs to be mounted
23 |              └── config2.json
```

Note: Anything commit/push to v1.yaml or v2.yaml, will trigger the deployment.

MANIFEST REPOSITORY STRUCTURE:

```
1 [cluster-name]-[cluster-type]_manifests/ # Custom name is also allowed
2 <namespace-branch>
3 |— namespace-configs/
4 |   |— [namespace].yaml           # Namespace configuration
```

STEP 1:

Expand step 1:

Create .github/workflows/cd.yaml file at the root of your repo with the following snippet.

```
1 name: App Deployment
2
3 on:
4   workflow_dispatch:
5   push:
6     branches: [main]
7     paths: ['**/values/v1.yaml']
8
9 permissions:
10   id-token: write
11   contents: write
12   pull-requests: write
13
14 jobs:
15   detect-changes:
16     runs-on: cvs-linux-self-hosted //self-hosted, for pcw org.
17     outputs:
18       CHANGED_ENVIRONMENTS: ${ steps.set-
19 matrix.outputs.CHANGED_ENVIRONMENTS }}
20   steps:
21     - uses: actions/checkout@v4
22       with:
23         fetch-depth: 0
24
25     - name: Detect changes
26       id: changes
27       uses: dorny/paths-
28 filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
29       with:
30         base: ${ github.ref_name }
31         filters: |
32           test-dev_v1: '**/deploy-configs/test-
33 dev/values/v1.yaml'
34           test-prod_v1: '**/deploy-configs/test-
35 prod/values/v1.yaml'
36
37     - name: Build Environment Matrix
38       id: set-matrix
39       run: |
40         ENVS=()
41
42         if [ "${ steps.changes.outputs.test-dev_v1 }" = "true"
43 ]; then
44           ENVS+=("${ ENV_NAME }:"test-
45 dev","HELM_FILE_VERSION":"v1")
46         fi
47
48         if [ "${ steps.changes.outputs.test-prod_v1 }" = "true"
49 ]; then
```

```

43     ENVS+=("{\"ENV_NAME\":\"test-
prod\", \"HELM_FILE_VERSION\":\"v1\"}")
44     fi
45
46     if [ ${#ENVS[@]} -gt 0 ]; then
47         JSON=$(IFS=,; echo "${ENVS[*]}")
48         echo "CHANGED_ENVIRONMENTS=$JSON" >> $GITHUB_OUTPUT
49     else
50         echo "CHANGED_ENVIRONMENTS=[]" >> $GITHUB_OUTPUT
51     fi
52
53     deploy:
54         uses: cvs-health-source-code/gitops-cd-
workflow/.github/workflows/helm-deploy.yaml@v1
55         needs: detect-changes
56         if: needs.detect-changes.outputs.CHANGED_ENVIRONMENTS != '[]'
57         strategy:
58             fail-fast: false
59             matrix:
60                 environment: ${ fromJson(needs.detect-
changes.outputs.CHANGED_ENVIRONMENTS) }
61             secrets: inherit
62             with:
63                 app-name: "test-app"
64                 environment: ${ matrix.environment.ENV_NAME }
65                 helm-template-repo: ${ vars.HELM_TEMPLATE_REPO }
66                 helm-file-version: ${ matrix.environment.HELM_FILE_VERSION
}}
67             helm-template-branch: ${ vars.HELM_TEMPLATE_BRANCH }
68             canary-weight: "0"
69             lob: "devex"
70             change-description: "test change"
71             rfc-short-desc: "test-description"
72             change-domain: "Health Care Delivery(HCD)"
73             change-impacted-website: "CVS.COM"

```

**** For PCW org. use the following in cd.yml**

```

1 jobs:
2   detect-changes:
3     runs-on: self-hosted

```

And:

```

1 deploy:
2   uses: cvs-health-pcw-source-code/gitops-cd-
workflow/.github/workflows/helm-deploy.yaml@main

```

You can now edit few parameters in it, which works according to your micro-service/deployment.

1. The app-name is given as test-app, which should be replaced with your micro-service name.
2. test-dev and test-prod are used as an example namespaces, which should be corrected accordingly and add additional namespaces if required.
3. canary-weight will remain "0", unless if any deployment requires traffic split between live version and deploy version.
4. The lob value changes from team to team, this parameter decides the respective approval group for the production deployment.

✓ Expand this for the list of lobes and respective prod approval group name

account: "account-gha-cd-approver"
retail: "retail-gha-cd-approver"
pharmacy: "pharmacy-gha-cd-approver"
sre: "pharmacy-gha-cd-approver"
health: "health-gha-cd-approver"
messaging: "martech-gha-cd-approver"
iot: "devex-gha-cd-approver"
ciam: "devex-gha-cd-approver"
activehealth: "martech-gha-cd-approver"
devex: "devex-gha-cd-approver"
ivr: "ivr-gha-cd-approver"

Note: These approval groups are [GitHub teams names](#) (for pcw, please search under pcw org.), the respective person should be present in that teams group to approve there pipeline during production deployment, if someone is not present, PROD-SUPPORT TEAM can add members to it.

5. change-description and rfc-short-desc can be ignored for non prod deployments but, when the micro-service is going for the prod release, the respective team has to update these parameters first with proper release notes, and then trigger the deployment next (any push/commit v1.yaml will trigger the deployment).
6. Below are the list of change-domains and impacted sites, if you have multiple, please chose one among them.

change-domain	change-impacted-website
Enterprise	♥ CVS - Online Drugstore, Pharmacy, Prescriptions & Health Information
Health Care Business(HCB)	♥ CVS Caremark Home
Lower Level Environment(LLE)	http://AETNA.COM
Pharmacy Services(PS)	SPECIALTY
Pharmacy & Consumer Wellness(PCW)	♥ View Health Records and Find Care CVS
Health Care Delivery(HCD)	myactivehealth.com

STEP 2:

▼ Expand step 2:

Create deploy-configs folder to host env./namespace specific helm values/configs for the deployment.

1. Under the deploy-configs folder, we first create common/common.yaml, this holds the common configs for the deployment irrespective of the environments/namespaces.

Below is the sample snippet:

```
1 #app related data
2 app:
3   name: test-app //update with your app name
4   port: 12345 //update with your port number
5   gitRepoName: cvs-health-source-code/test-repo //update with your
   repo path
6   enableOpenTelemetry: "true" //optional can be commented if not
   required.
7
8
9 #livenessProbe details
10 livenessProbeDetails:
11   probePath: "/microservices/test-app/healthz" //update with your
   probe path
12   initialDelaySeconds: 30
13
14 #readinessProbe details
15 readinessProbeDetails:
16   probePath: "/microservices/test-app/healthz" //update with your
   probe path
17   initialDelaySeconds: 30
18
19 #startupProbe details
20 startupProbeDetails:
21   probePath: "/microservices/test-app/healthz" //update with your
   probe path
22   initialDelaySeconds: 60
23   periodSeconds: 10
24   failureThreshold: 30
25
26 #custom pod annotations //optional this section can be commented
   if not required.
27 customPodAnnotations: |
28   name: "{{ .Values.app.name }}"
29   prometheus.io/port: "{{ .Values.app.port }}"
30   prometheus.io/path: "/actuator/metrics"
31   instrumentation.opentelemetry.io/inject-java: "opentelemetry-
   operator/my-instrumentation"
32   instrumentation.opentelemetry.io/container-names: "{{
   .Values.app.name }}"
33
34 istioHttp: |
35   {{- if .Values.istio.cookieBasedRouting }}
36   # Cookie-based routing: Route canary=on requests to new
   deployment
37   - match:
38     - headers:
39       cookie:
40         regex: ^(.?;)?\s*(canary=on)(;.*)?$
41       uri:
42         prefix: /depservices/
43     route:
44   - destination:
45     host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
   }}.svc.cluster.local
46     port:
47       number: {{ .Values.app.port }}
48       subset: {{ regexReplaceAll "\\W+"
   .Values.istio.canaryAppVersion "-" }}
49     weight: 100
50   {{- end }}
51   # Default routing for all scenarios (cookie-based default,
   weight-based, rolling)
```

```

52 - match:
53   - uri:
54     prefix: /depservices/
55     route:
56       {{- if and (ne .Values.istio.currentAppVersion
57         .Values.istio.canaryAppVersion) (not
58         .Values.istio.cookieBasedRouting) }}
59       # Weight-based canary: Split traffic between current and canary
60       - destination:
61         host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
62           }}.svc.cluster.local
63         port:
64           number: {{ .Values.app.port }}
65           subset: {{ regexReplaceAll "\\W+"
66             .Values.istio.currentAppVersion "-" }}
67         weight: {{ .Values.istio.prodWeightage }}
68       - destination:
69         host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
70           }}.svc.cluster.local
71         port:
72           number: {{ .Values.app.port }}
73           subset: {{ regexReplaceAll "\\W+"
74             .Values.istio.canaryAppVersion "-" }}
75         weight: {{ .Values.istio.canaryWeightage }}
76       {{- else }}
77       # Single version routing (cookie-based default OR rolling
78       deployment)
79       - destination:
80         host: {{ .Values.app.name }}.{{ .Values.appData.nameSpace
81           }}.svc.cluster.local
82         port:
83           number: {{ .Values.app.port }}
84           subset: {{ regexReplaceAll "\\W+"
85             .Values.istio.currentAppVersion "-" }}
86         weight: 100
87       {{- end }}

```

2. Now its time to create namespace specific folder and respective values and config folders under it, if your app do not require any additional configs to be passed then ignore creating config folder.

Sample v1.yaml:

```

1 #namespace, app version and image details...
2 appData:
3   nameSpace: test-dev //update correct one
4   dockerHub: cvsh.jfrog.io
5   dockerImageGroupName: cvsdigital-docker/ghemu/... //update
6   correct one
7   dockerImageName: test-app //update correct one
8   dockerImageTag: 1.0.0 //update correct one
9
10 #hpa settings - min and max replicas and minAvailable for PDB
11 hpaSettings:
12   minAvailable: 1
13   minReplicas: 1
14   maxReplicas: 1
15   averageUtilization: 80
16
17 #Updates domain portal //remove this section, if not required
18 # deployValidations:
19 #   - name: i90-domain
20 #     repo_name: i90-domain-harness-automation-deploy
21
22 # Pod resource requests and limits
23 resources:
24   requests:

```

```

24     cpu: "100m"
25     memory: "250Mi"
26     limits:
27       cpu: "200m"
28       memory: "350Mi"
29
30 # environment variables for app - secret values come from vault
31 envVars: |
32   - name: TEST_KEY
33     value: TEST_VALUE
34   - name: TEST_KEY
35     value: TEST_VALUE
36
37 #configMap BasePath //remove this section, if not required
38 configBasePath: /opt/digital/... //update correct one
39
40 # ConfigMaps to be Created //remove this section, if not required
41 configMap:
42   test-app-application.yaml: |
43     {{- .Files.Get "config/test-app-application.yaml" }}
44   test-app-error.json: |
45     {{- .Files.Get "config/test-app-error.json" }}
46
47 # env secrets to be referred as ENV variabes //remove this section,
48 if not required
49 envSecrets: |
50   - secretRef:
51     name: secret-folder-name1
52     optional: false
53   - secretRef:
54     name: secret-folder-name2
55     optional: false
56
57 extraVolumeMounts: | //remove this section, if not required
58   - name: cert-folder-name
59     mountPath: /opt/digital/microservices/test-app/certs
60     readOnly: true
61
62 extraVolumes: | //remove this section, if not required
63   - name: cert-folder-name
64     secret:
65       secretName: cert-folder-name
66       optional: false
67
68 # Cloud sql sidecar enabled //remove this section, if not required
69 or fill in correct values
70 cloudSqlSidecar:
71   enabled: true
72   instanceName: digital-dev-iot:us-east4:test-postgresql-
73   dev=tcp:4321
74   requestCpu: "500m"
75   requestMemory: "1Gi"
76   limitCpu: "500m"
77   limitMemory: "1Gi"
78
79 # Cloud sql sidecar enabled //remove this section, if not required
80 or fill in correct values
81 workloadIdentity:
82   gcpProject: digital-dev-iot-test
83   serviceAccountName: cloudsql-digital-iot-test
84   serviceAccountEmail: cloudsql-digital-iot@digital-dev-test-
85   iot.iam.gserviceaccount.com

```

3. If you have mentioned configs in your v1.yaml then create config folder accordingly and have the respective files under it.

STEP 3:

▼ Expand step 3:

1. Create manifest repo if does not exists (add a readme file - optional/recommended).

We recommend to create manifest repo with the following naming standard:

[cluster-name]-[cluster-type]_manifests

eg: pharma-dev-gke_manifests, prod-pharmacy-rke_manifests

Note:

1. Manifest repo will be unique to the cluster, we do not recommend multiple manifest repos to one cluster. All the namespaces under that cluster, will be as branches under the manifest repo.
 2. Custom namespaces are also allowed for naming manifest repo, but it has to be passed as an additional parameter for the deployment pipeline, which will be discussed down the line.
2. Cut a branch from **main** in the manifest repo with the namespace name and create **namespace-configs/[namespace].yaml** file in it.

sample content:

```
1 #argocd related data
2 argo:
3   project: default
4   manifestRepo: "https://github.com/cvs-health-source-code/test-
5   manifest.git" //update repo url .git
6   notifyEmailList: "abc@cvshealth.com, user2@cvshealth.com"
7
8 #istio related values
9 istioInternalHosts: | // update your cluster internal dns values
10   - "sit-dgtl-pbm.internal.cvshealth.com"
11   - "other-internal-dns"
12
13 istioExternalHosts: | // update your cluster external dns
14   values
15   - "external-dns.cvshealth.com"
16
17 #istio Host details for Istio auth // update the list
18 istioHosts: '["sit-dgtl-pbm.internal.cvshealth.com", "other-
19   internal-dns", "external-dns.cvshealth.com"]'
20
21 #namespace resources default and limits
22 namespaceDetails:
23   defaultMemory: "350Mi"
24   defaultCpu: "500m"
25   maxMemory: "2Gi"
26   maxCpu: "2"
27
28 jwtVaultClusterScope: // optional, remove this section if not
29   required
30   vaultURL: "https://vault-nonprod.cvshealth.com"
31   tenant: "gcp/rke-project-name"
32   kubernetesCluster: "gcp/rke-cluster-name"
33   externalSecrets:
34     i90-secrets:
35       path: kv-v2/app/i90-apollo/data/test
36
37 #vault Details at namespace level
38 jwtVault: // optional, remove this section if not required
39   vaultURL: "https://vault-nonprod.cvshealth.com"
40   tenant: "gcp/rke-project-name"
41   kubernetesCluster: "gcp/rke-cluster-name"
```



```

38 externalSecrets:
39   - secret-folder-name1
40   - secret-folder-name2
41   - secret-folder-name3
42 externalSecretsBase64:
43   - cert-folder-name1

```

STEP 4:

✓ Expand step 4:

Now go to setting of your source-code repo (one used in step 1 & 2), if you do not see setting over your repo that mean you do not have admin access over your repo.

1. Navigate to settings/ Secrets and variables (left-side)/ Actions:

Under Repository secrets, please add your cluster argo admin password by clicking on new repository secret.

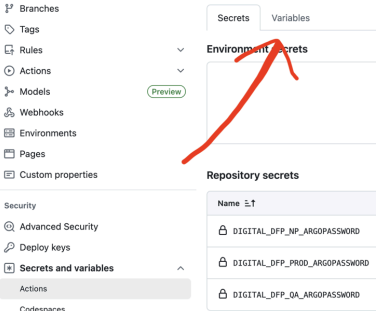
Eg: Lets say I am trying to deploy to a dev namespace, which is/should-be part of **pharma-dev** cluster, then I create a secret with key as **PHARMA_DEV_ARGOPASSWORD** and value is your argo admin password.

Note: For Prod, its prod support team who will add it, for the moment, and maintain same format of naming as in example screen shots.

Repository secrets New repository secret

Name ↕	Last updated
DIGITAL_DFP_NP_ARGOPASSWORD	3 weeks ago  
DIGITAL_DFP_PROD_ARGOPASSWORD	2 weeks ago  
DIGITAL_DFP_QA_ARGOPASSWORD	3 weeks ago  

2. Now hover to variables section and add your cluster argo admin user name under repository variables section.



Repository secrets New repository secret

Name ↕	Last updated
DIGITAL_DFP_NP_ARGOPASSWORD	3 weeks ago  
DIGITAL_DFP_PROD_ARGOPASSWORD	2 weeks ago  
DIGITAL_DFP_QA_ARGOPASSWORD	3 weeks ago  

Repository variables New repository variable

Name ↕	Value	Last updated
DIGITAL_DFP_NP_ARGOUSER	admin	3 weeks ago  
DIGITAL_DFP_PROD_ARGOUSER	admin	2 months ago  
DIGITAL_DFP_QA_ARGOUSER	admin	3 weeks ago  

3. Now Click on Environments to your left and create your namespaces under it (delete if there is any unwanted) with following parameters under it.

Note: When you create your environment/namespace, fill in the environment variables section with CLUSTER_NAME & CLUSTER_TYPE which are mandatory.

Please follow as in example screen shots:

The screenshot shows the GitHub Actions 'Environments' page. On the left is a sidebar with navigation options: General, Access, Collaborators and teams, Team and member roles, Code and automation, Branches, Tags, Rules, Actions, Models, Webhooks, Environments (selected), Pages, and Custom properties. The main content area is titled 'Environments' and includes a 'New environment' button. Below this is a table listing environments: gha-test-qa (3 variables), gha-test-dev (2 variables), gha-test-prod-dr (3 variables), and gha-test-prod (3 variables). Below the table is the 'Environment variables' section, which includes an 'Add environment variable' button and a table of variables for the selected environment.

Name	Value	Last updated
CLUSTER_NAME	digital-dfp-np	3 weeks ago
CLUSTER_TYPE	GKE	3 weeks ago

Additionally you can add few other variables such as:

MANIFESTS_REPO_NAME - incase you have custom manifest repo name.

GITOPS_SUCCESS_DEPLOY_WAIT_TIME - incase you want pipeline to wait for more time, in the argo sync and app health validation section. (default is 600 seconds which should be good, you can custom entry values in seconds if you want it more - not recommended)

NEXT_ENV - incase you want the automatic deployment trigger in other namespace.

(eg: Lets say I updated my docker image tag in dev namespace v1.yaml file then deployment will get automatically trigger for dev and if its successful, since I added NEXT_ENV value as qa, then the pipeline will update the same docker image tag in qa namespace v1.yaml which triggers qa deployment. It only updates docker image tag, not the rest of configs)

GITOPS_SUCCESS_DEPLOY_WAIT_TI...	800	2 months ago		
MANIFESTS_REPO_NAME	gke_sit-ui-gke_manifests	2 months ago		
NEXT_ENV	blocks-sit2	now		

STEP 5:

Expand step 5:

Its all done, now any trigger to your v1.yaml or v2.yaml (optional) will trigger the deployment.

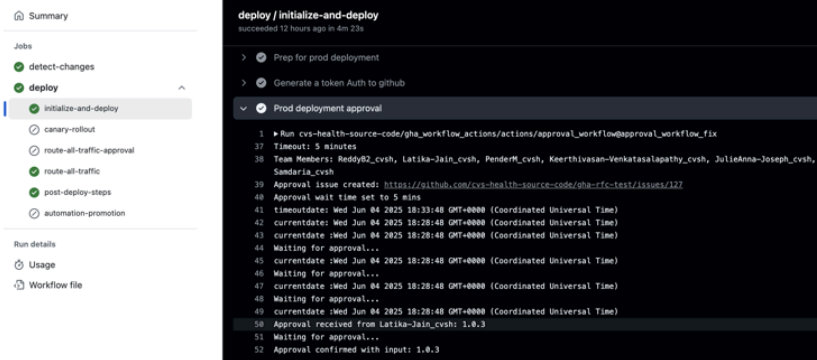
For Vault configurations:

[Using Vault with Harness via native JWT](#)

Production pipeline approval process:

✓ Expand Production pipeline approval process:

Unlike the non-prod deployments the pipeline will wait for 5 minutes (it can not be increased) at Prod deployment approval stage, for pipeline to receive approval, else pipeline aborts.



We advise you to have the pipeline being monitored, as at this stage there will be an GithubIssue for approval created and published as a [https://](#) link.

That link has to be passed to the respective member who has access to approve this prod deployment.

To approve this deployment, the respective member as to comment **approve keyword** along with deploy version.

Example:

Since the deploy version in here is 1.0.3, the member authorized to approve has commented **approve 1.0.3**

Production Rolling deployment for test-app version 1.0.3 to gha-test-prod #127

Closed



gitemudevopsteam opened 13 hours ago

test-app Production Rolling deployment to environment gha-test-prod

LIVE_VERSION: 1.1.6 - current image running in gha-test-prod
DEPLOY_VERSION: 1.0.3 - to be deployed into gha-test-prod
PERFORM_CLEANUP: Yes - if yes, after new image is deployed, 1.1.6 will be deleted
TRAFFIC_ROUTING: 100% traffic will be routed to 1.0.3
GITHUB_APP_REPO: - Deployment status is posted to this repo .

DEPLOYMENT_TRIGGERED_BY:ReddyB2_cvsh

APPROVAL_GROUP: devex-gha-cd-approver

!! ATTEN !!

To approve ☒ the deployment, comment:approve e.g., approve v1.2.3 and close the issue.

To reject ☒ the deployment comment:reject e.g., reject v1.2.3 and close the issue.

CHANGE_DOMAIN:Health Care Delivery(HCD)

CHANGE_IMPACTED_WEBSITE:CVS.COM

☒ Approvers: ReddyB2_cvsh, Latika-Jain_cvsh, PenderM_cvsh, Keerthivasan-Venkatasalapathy_cvsh, JulieAnna-Joseph_cvsh, Venkata-Kondapalli_cvsh, Harish-Kuchana_cvsh, Amber-Samdaria_cvsh

! ReddyB2_cvsh cannot approve this workflow.

To approve, comment:

approve <input> (e.g., approve v1.2.3)

Create sub-issue

gitemudevopsteam added manual-approval 13 hours ago



Latika-Jain_cvsh 13 hours ago

approve 1.0.3

Caution:

1. Please add path-ignore section in your ci.yml workflow if you have one, in your repo. So that for every CD specific update the CI pipeline will not trigger.
2. This will not work for the mono repo style, meaning one repo hosting multiple micro-service deployments across multiple environments, the process is almost the same with little change (will publish a doc on it).
3. For teams migrating from **Harness**, **do not use the existing manifest repo or namespaces**.
4. Make sure you have your cluster listed in here:

https://github.com/cvs-health-source-code/stargate_allow_list/blob/main/cd/gha_cd/cd_env_config.yaml (source code org.)

https://github.com/cvs-health-pcw-source-code/pcw_allow_list/blob/main/cd/gha_cd/cd_env_config.yaml (pcw org.)

Create new manifest repo and deploy your micro-services to new namespace and once the deployment is validated in new namespace, then the respective references in the old-namespaces can be delete. Just an fyi, new namespace comes up with new vault engine

[#pcw-devex-devsecops-engagement-team](#)